Chapter 7: Malware Detection with Machine Learning

Lectured by Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

Duration: 90 minutes

Objective: Provide an overview of malware, defensive detection & prevention methods, and a practical machine-learning pipeline (static PE features → feature selection → model training/evaluation → CFG→DNA string approach → clustering/classification).

Lecture Outline

1. Malware overview & illustrative examples
2. Detection & prevention (operational view)
3. Machine-learning approaches to malware detection
4. Static PE feature extraction (practical)
5. Feature selection strategies
6. Model training, validation, and evaluation
7. CFG → DNA string: concept and extraction pipeline
8. Clustering and classification approaches
9. Lab suggestions, ethics, and datasets

1. Malware — overview and examples

Definition: Software designed to disrupt, damage, or gain unauthorized access to computer systems.

High-level families:

- **Virus:** attaches to host files, spreads on execution.
- **Worm:** self-propagates across networks.
- **Trojan:** appears benign but hides malicious payload.
- **Ransomware:** encrypts data for ransom.
- **Spyware/Keylogger:** stealthy data exfiltration.
- **Botnet clients (C2):** enable distributed attacks.

1. Malware — Overview and Taxonomy

Why it matters (detection goals):

- Identify **what** (family/capability)
- Understand **how** (delivery, execution, persistence)
- Infer **why** (objective: espionage, crime, sabotage)
- Guide **response** (containment, eradication, recovery)

Core Malware Lifecycle (High-Level)

1. **Initial Access** → 2) **Execution** → 3) **Persistence**
2. **Privilege Escalation** → 5) **Defense Evasion** → 6) **Discovery**
3. **Credential Access** → 8) **Lateral Movement** → 9) **C2**
4. **Collection** → 11) **Exfiltration / Impact**

Taxonomy at a Glance

Family	Primary Goal	Typical Delivery	Common Signals
Virus	Propagate via infected files	Removable media / legacy file-sharing	Modified PE sections, file anomalies
Worm	Self-spread across networks	Exploit weak services	Surge in scanning, unusual connections
Trojan	Deception + backdoor	Phishing/attachments, drive-by	New services, outbound C2 beacons

Taxonomy at a Glance

Family	Primary Goal	Typical Delivery	Common Signals
Ransomware	Encrypt & extort	Phishing, RDP brute-force	Mass file renames, note files, CPU spikes
Spyware/Keylogger	Steal data/creds	Bundled with trojans	Hooks to input APIs, exfiltration bursts
Rootkit/Bootkit	Stealth & persistence	Post-compromise	Hidden drivers, tampered kernel/boot
Botnet (C2)	Remote control at scale	Trojan → C2 enroll	Periodic DNS/TLS beacons, DGA domains
Fileless	Live-off-the-land	Script/macros, WMI	Powershell/WMI logs, no disk artifacts

Viruses (File Infectors)

Behavior: Attach to executables/documents; replicate on execution.

Techniques: Entry-point obfuscation, section injection, polymorphism.

Indicators:

- PE header anomalies, unexpected code caves
- Hash mismatch after normal app updates

  Defense: Application allow-listing, integrity monitoring, AV heuristics.

Worms (Network Self-Propagation)

Behavior: Exploit network/services to auto-spread.

Indicators:

- Port scanning bursts, SMB/RPC anomalies, traffic spikes
- Sudden multi-host infections within a subnet
  Defense: Patch hygiene, network segmentation, east-west traffic monitoring, rate-limiting.

Trojans, Backdoors & RATs

Behavior: Masquerade as benign; install backdoor / remote admin tool.

Capabilities: Keylogging, screen capture, file ops, command execution.

Indicators:

- New autoruns, abnormal parent-child process trees
- Outbound C2 over HTTP(S)/TLS, often to newly registered domains

  Defense: Email/web filtering, EDR behavior rules, DNS sinkholing, zero-trust access.

Ransomware (Crypto & Locker)

Behavior: Encrypt files (crypto) or block system (locker); demand payment.
Pre-encryption stage: Recon, credential theft, AD discovery, data exfiltration (“double extortion”).
Indicators:

- Rapid file renaming, writes of ransom notes, shadow copy deletion
- Lateral movement (RDP/SMB), peak write I/O
  Defense: MFA, disable SMBv1, least privilege, immutable backups, EDR canary files, rapid isolation playbooks.

Spyware, Keyloggers, Info-Stealers

Behavior: Harvest credentials, cookies, autofill, crypto wallets.

Channels: Phishing attachments, malicious installers, cracks.

Indicators:

- Hooks into input APIs, browser data access, frequent small HTTPS posts

  Defense: Browser hardening, password managers with phishing protection, egress filtering.

Rootkits & Bootkits (Stealth Persistence)

Behavior: Hide processes/files; intercept kernel calls; survive reboots (bootkits).

Indicators:

- Discrepancies between user-mode vs. kernel-mode listings
- Unsigned drivers; modified boot records
  Defense: Secure Boot, driver signing enforcement, kernel telemetry, trusted re-imaging.

Botnets & C2 Ecosystems

Behavior: Enroll hosts into a remotely controlled network.
C2 Patterns: Centralized (HTTP/TLS), P2P, fast-flux, DGA domains.
Indicators:

- Periodic beaconing (regular intervals), TLS to rare SNI/JA3 fingerprints
  Defense: DNS filtering, JA3/JA3S analytics, anomaly-based beacon detection, egress allow-lists.

Fileless & Living-off-the-Land (LotL)

Behavior: Use native tools (PowerShell, WMI, rundll32) and memory-only payloads.

Indicators:

- Script block logs, constrained language mode bypass attempts
- Long-running PowerShell with encoded commands
  Defense: PowerShell logging (ScriptBlock/Module), AMSI integration, block unsigned macros, WDAC/AppLocker.

Adware, PUP/PUA, Cryptominers, Wipers, Logic Bombs

- **Adware/PUP:** Unwanted ads/toolbars; privacy risk, lateral risk.
- **Cryptominers:** High CPU/GPU, outbound pools, stealth persistence.
- **Wipers:** Destructive overwrite (availability impact).
- **Logic bombs:** Time/condition-based triggers.
  Defense: Application control, resource monitoring, SIEM correlation for unusual CPU/network use.

Supply-Chain, Loader/Dropper & Staged Malware

- **Supply-chain:** Compromised vendor updates/libraries.
- **Loaders/Dropper:** Small initial stage fetches heavy payload later.
  Indicators:
- Update process anomalies, odd signer metadata, unexpected outbound fetches
  Defense: SBOM validation, code-signing verification, EDR network controls, allow-listing update endpoints.

Mobile, IoT, and OT Malware

- **Mobile (Android/iOS):** Malicious apps, SMS fraud, spyware (permissions misuse).
- **IoT:** Default creds, weak update channels, botnets for DDoS.
- **OT/ICS:** Protocol abuse (Modbus, DNP3), high impact on safety & availability.
  Defense: MDM, device posture checks, IoT/OT network segmentation, protocol-aware monitoring.

Propagation, Persistence, Evasion — Cross-Family Techniques

- **Propagation:** Phishing, RDP brute force, exploit kits, USB, supply-chain.
- **Persistence:** Autoruns, scheduled tasks, registry run keys, services, WMI event consumers.
- **Evasion:** Packing/obfuscation, signed binaries, LOLBins, sandbox/VM checks, encryption, DGA.

Analyst Tip: Track these *capabilities* in addition to “family names” for durable detections.

Indicators of Compromise (IOC) vs. Indicators of Behavior (IOB)

Type	Examples	Pros	Cons
IOC	Hashes, domains, IPs, file paths	Precise, fast blocking	Fragile to change
IOB	TTP sequences, process trees, API call patterns	Robust to variants	Harder to engineer

Balanced strategy: IOC for quick wins + IOB for resilience.

Detection & Prevention (Per Family)

Family	Host Signals	Network Signals	Key Controls
Worm	New services, autoruns	Scans, surge in connections	Patch, segment, IDS rules
Trojan/RAT	New processes, persistence	Periodic C2 beacons	EDR, DNS/TLS analytics
Ransomware	Mass file ops	Pre-encryption C2/exfil	Backups, EDR canaries, MFA

Detection & Prevention (Per Family)

Family	Host Signals	Network Signals	Key Controls
Spyware	Input hooks, browser access	Small periodic posts	Browser hardening, DLP
Rootkit	Hidden modules	Odd handshake patterns	Secure Boot, kernel attestation
Fileless	PowerShell/WMI abuse	Minimal footprints	Logging + WDAC/AppLocker

2. How to detect and how to prevent (

Preventive controls:

- Endpoint protection (AV/EDR), secure configuration, timely patching.
- Least privilege, application allow-listing, secure email gateways.
- Network segmentation, web/email filtering, sandboxing suspicious samples.

2. How to detect and how to prevent (

Detection controls:

- Signature-based detection (fast, low false positives on known samples).
- Heuristic / rule-based detection.
- Behavioural/telemetry detection (EDR, process/network monitoring).
- ML-based static & dynamic detection (useful for unknown or obfuscated samples).

3. Malware detection using machine learning

Why ML?

- Generalise beyond signatures, adapt to obfuscation and polymorphism.
- Scale to large corpora and multi-feature telemetry.

Two main ML paradigms for malware detection:

- **Static analysis:** extract features from file binaries (PE headers, imports, entropy, strings) — no execution. Safe, fast, but evasion possible.
- **Dynamic analysis:** derive behavioural features from sandbox runs (API calls, network flows) — more robust but resource-heavy and potentially evasive if malware detects sandboxing.

Typical pipeline: Feature extraction → preprocessing → feature selection → model training → evaluation → deployment.

4. Feature extraction from PE file — static based

Common static features (PE Windows binaries):

1. **PE Header fields**
   - `NumberOfSections`, `TimeDateStamp`, `Machine`, `SizeOfImage`, `Characteristics`, `Subsystem`.
2. **Section characteristics**
   - Names (.text, .rdata, .data), sizes, entropy per section (high entropy ⇒ packed/obfuscated).
3. **Imported functions & DLLs**
   - Presence/absence of suspicious imports (e.g., network, process, registry APIs), import counts, API families.

4. Feature extraction from PE file — static based

Common static features (PE Windows binaries):

4. **Exported symbols**
    - Export table presence and names (typical for DLLs).
5. **Strings**
    - URLs, IPs, suspicious keywords, command patterns.
6. **Byte-level features**
    - N-grams of raw bytes (e.g., 1–4 byte n-grams), byte histograms.

4. Feature extraction from PE file — static based

Common static features (PE Windows binaries):

7. **Resource and certificate info**
    - Presence/absence and issuer metadata.
8. **File metadata**
    - File size, entropy, digital signature presence.

Practical extraction toolkits: `pefile`, `lief`, `capstone` (for disassembly), `scikit-learn` for vectorisation. (In class use sanitized/benign test files or curated datasets.)

5. Example static feature extraction

```
# Defensive example — extract imports and section entropy
# Requires: pip install pefile numpy

import pefile, math, numpy as np

def entropy(data: bytes) -> float:
    if not data:
        return 0.0
    counts = np.bincount(np.frombuffer(data, dtype=np.uint8), minlength=256)
    probs = counts / counts.sum()
    probs = probs[probs > 0]
    return -np.sum(probs * np.log2(probs))
```

5. Example static feature extraction

```
def extract_pe_features(path):
    pe = pefile.PE(path, fast_load=True)
    pe.parse_data_directories(directories=[pefile.DIRECTORY_ENTRY['IMAGE_DIRECTORY_ENTRY_IMPORT']])
    features = {}
    # Imports
    imports = []
    if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
        for entry in pe.DIRECTORY_ENTRY_IMPORT:
            dll = entry.dll.decode(errors='ignore').lower()
            funcs = [imp.name.decode(errors='ignore') if imp.name else b'' for imp in entry.imports]
            imports.append((dll, len(funcs)))

    features['num_imports'] = sum(c for _, c in imports)
    features['imported_dlls'] = [d for d,_ in imports]
    # Sections entropy
    features['section_entropy'] = {sec.Name.decode().strip('\x00'): entropy(sec.get_data()) for sec in pe.sections}
    return features

# Example usage (use only on test binaries):
# feats = extract_pe_features('benign_sample.exe')
# print(feats)
```

6. Feature selection

Why select features?

- Reduce dimensionality, improve generalisation, remove noisy/redundant features.

Categories of feature selection:

- **Filter methods:** correlation, chi-square, mutual information, variance threshold. Fast, independent of model.
- **Wrapper methods:** recursive feature elimination (RFE) with cross-validation. More accurate, expensive.
- **Embedded methods:** feature importance from tree models (Random Forest, XGBoost) or L1 regularisation (Lasso).

6. Feature selection

Practical approach:

1. Start with domain filters (drop constant or near-constant features).
2. Use tree-based importance to shortlist.
3. Validate with RFE and cross-validation.

7. Model training and evaluation

Standard workflow:

1. Split dataset: train / validation / test (e.g., 60/20/20) or use stratified k-fold CV.
2. Preprocess: standardise or normalise numeric features; encode categorical features (one-hot or ordinal).
3. Train candidate models: Random Forest, XGBoost, LightGBM, SVM, Logistic Regression, Neural Networks.
4. Hyperparameter tuning: grid search or Bayesian optimisation (with CV).
5. Evaluate on hold-out test set.

7. Model training and evaluation

Evaluation metrics (malware detection):

- **Precision** = TP / (TP + FP)
- **Recall (TPR)** = TP / (TP + FN) — critical to not miss malware
- **F1 score** — harmonic mean of precision & recall
- **ROC AUC / PR AUC** — for threshold-independent performance
- **Confusion matrix** — inspect FP/ FN tradeoffs
- **Detection time / resource usage** — operational constraints

Note: In imbalanced datasets, emphasise precision/recall and PR-AUC.

8. Example ML pipeline (scikit-learn) — training & evaluation

```
# Defensive ML pipeline example (vectorised features)
# pip install scikit-learn pandas

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd

# df: pandas DataFrame with features and 'label' (0 benign, 1 malicious)
X = df.drop(columns=['label'])
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.2, random_state=42)

rfc = RandomForestClassifier(n_estimators=200, class_weight='balanced', random_state=42)
rfc.fit(X_train, y_train)
y_pred = rfc.predict(X_test)

print(classification_report(y_test, y_pred, digits=4))
print(confusion_matrix(y_test, y_pred))
```

Practical notes: track model drift, retrain with fresh samples, and integrate explainability (feature importances, SHAP) to justify detections.

9. Feature Extraction from PE file — CFG base to create DNA string (concept)

Motivation: Static opcode sequences and CFG structures capture program control behaviour resilient to some obfuscations.

High-level idea:

1. Disassemble PE to basic blocks (use Capstone or angr).
2. Build the Control Flow Graph (CFG): nodes = basic blocks; edges = control transfers (calls, jumps).
3. Linearise CFG into sequences (walks) — e.g., depth-first traversals, or extract ordered sequences per function.

9. Feature Extraction from PE file — CFG base to create DNA string (concept)

4. Map basic block signatures or opcode mnemonics to a compact alphabet (A, C, G, T style) → produce a **DNA-like string** for each binary.
5. Apply sequence analysis techniques: k-mer frequency (n-gram counts), sequence kernels, or feed to sequence models (RNN, Transformers).

Why "DNA"? Analogous to biological sequences — allows use of bioinformatics tools (k-mer analysis, edit distance) for similarity/clustering.

10. Practical algorithm: CFG → DNA string (outline)

1. **Disassemble** binary safely (offline) to get functions and basic blocks.
2. **For each basic block:** compute a short signature (e.g., hash of opcodes or top-k opcodes).
3. **Map signature → alphabet symbol:** H = hash % 4 (or 16), map to A,C,G,T (or extended alphabet).
4. **Traverse CFG:** produce ordered string per function and concatenate for whole binary.
5. **Vectorise:** compute k-mer counts for k=3..6 or use embedding (word2vec on opcode sequences).
6. **Use features** for clustering/classification.

Safety note: Do not execute the binary; use disassembly only in an isolated analysis environment.

11. Example (pseudo) code for CFG → k-mer counts

```
# Pseudocode (conceptual). Use disassembly frameworks in isolated analysis VMs.
# 1) Disassemble and obtain basic block opcodes per function.
# 2) For each block, create a short token: e.g., opcode mnemonic sequence -> hash
# 3) Map hash -> small alphabet symbol
# 4) Walk function order -> produce string; compute k-mer counts

from collections import Counter

def block_to_symbol(opcodes):
    # simple deterministic mapping for classroom demo
    s = ''.join(opcodes)
    h = abs(hash(s)) % 256
    # map to 16-symbol alphabet
    alphabet = [chr(ord('A') + i) for i in range(16)]
    return alphabet[h % len(alphabet)]

def cfg_to_string(cfg):
    # cfg: dict mapping function -> list of blocks (each block -> opcode list)
    seq = []
    for func in cfg.functions:
        for block in func.blocks:
            sym = block_to_symbol(block.opcodes)
            seq.append(sym)
    return ''.join(seq)

def kmer_counts(seq, k=3):
    counts = Counter()
    for i in range(len(seq)-k+1):
        counts[seq[i:i+k]] += 1
    return counts
```

12. Clustering and classification approaches

Clustering (unsupervised):

- k-means on k-mer vectors (requires vector normalisation)
- DBSCAN for density-based grouping (finds outliers)
- Hierarchical clustering (dendrograms to inspect families)
- Spectral clustering for complex manifolds

12. Clustering and classification approaches

Classification (supervised):

- Tree ensembles: Random Forest, XGBoost, LightGBM — strong baselines for tabular features
- SVM (with string kernels for sequence features)
- Deep learning:
  - CNNs on byte-image representation (convert binary bytes into 2D image)
  - RNNs / Transformers on opcode or symbol sequences
- Hybrid approaches: combine static features + CFG sequence embeddings + dynamic telemetry

13. Model evaluation & deployment considerations

False positives create operational overhead — tune thresholds and provide human analyst review.
Explainability: use SHAP, LIME, or feature importances to present rationales to SOC analysts.
Adversarial robustness: attackers may obfuscate; use ensemble of static + dynamic features to increase resilience.
Data pipeline: ingest new labeled samples, retrain periodically, validate on recent malware families.

14. Datasets, tooling, and lab suggestions

Suggested (ethical) datasets for teaching / research:

- EMBER (public static PE features dataset) — good for ML baselines.
- Curated benign/malicious corpora from research datasets (use institutional research access and legal review).

Tooling:

- `pefile`, `lief` (static analysis)
- `capstone` / `radare2` / `ghidra` (disassembly)
- `angr` (CFG extraction, advanced static analysis)
- `scikit-learn`, `xgboost`, `lightgbm`, `tensorflow/keras` (ML)
- `pyshark` / `tshark` for dynamic network telemetry

14. Datasets, tooling, and lab suggestions

Lab idea:

- Students extract static features from a sanitized dataset, perform feature selection, train/test a Random Forest, evaluate metrics, and produce a short report.
- Advanced: implement CFG→DNA pipeline on a small benign set (no malware) to practise the technique.

15. Ethical, legal & safety considerations

- Handling malware samples requires isolated environments (VMs with no external network), snapshots, and institutional approvals.
- Classroom demos should use sanitized or synthetic samples; provide students with pre-computed features/PCAPs rather than raw binaries unless strict lab controls exist.
- Always record provenance, maintain chain of custody for samples used in research, and comply with local laws and university policies.

16. Summary & takeaways

- Static PE features provide fast, useful signals; dynamic features add robustness.
- Feature engineering (imports, entropy, strings, CFG sequences) is critical — ML performance depends more on features than on model choice.
- CFG→DNA string is a promising approach to transform control-flow structure into sequence features amenable to sequence analysis and clustering.
- Combine static and dynamic evidence, and prioritise safe, ethical lab practices.

Thank You

Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

chatchanan.ja@mail.wu.ac.th